Warsaw University of Technology

FACULTY OF
ELECTRONICS AND INFORMATION TECHNOLOGY

Institute of Institute of Radioelectronics and Multimedia Technology

Final project

Postgraduate diploma thesis
Deep Learning - Applications in Digital Media

Graph neural networks in medical knowledge graphs

Marta, Monika
student record book number XXXX

thesis supervisor
YYYYY

WARSAW 2026

Graph neural networks in medical knowledge graphs

Streszczenie.

Głównym celem pracy jest Nacisk w pracy został położony na ... W pierwszej części pracy przedstawiono ... Następnie opisano ... Druga część pracy rozpoczyna się od ... Eksperymenty opisane w dalszej części pracy przedstawiają ... Dodatkowym etapem pracy jest ... W zakończeniu pracy przedstawiono podsumowanie wykonanych działań oraz dalsze działania, które mogą przynieść poprawę ...

Słowa kluczowe: NFT, ChatGPT, Buzzword300

Graph neural networks in medical knowledge graphs

Abstract.

Summary in English.....

Keywords: ChatGPT, Buzzword300

załącznik nr 10 do zarządzenia
nr 46 /2016 Rektora PW

Politechnika Warszawska
Warsaw University of Technology

………......................
miejscowość i data
place and date

……………………………..
imię i nazwisko studenta
name and surname of the student

……………………………..
numer albumu
student record book number

………………….……….
kierunek studiów
field of study

OŚWIADCZENIE

DECLARATION

Świadomy/-a odpowiedzialności karnej za składanie fałszywych zeznań oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie, pod opieką kierującego pracą dyplomową.
Under the penalty of perjury, I hereby certify that I wrote my diploma thesis on my own, under the guidance of the thesis supervisor.

Jednocześnie oświadczam, że:
I also declare that:

- niniejsza praca dyplomowa nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. z 2006 r. Nr 90, poz. 631 z późn. zm.) oraz dóbr osobistych chronionych prawem cywilnym,
- *this diploma thesis does not constitute infringement of copyright following the act of 4 February 1994 on copyright and related rights (Journal of Acts of 2006 no. 90, item 631 with further amendments) or personal rights protected under the civil law,*
- niniejsza praca dyplomowa nie zawiera danych i informacji, które uzyskałem/-am w sposób niedozwolony,
- *the diploma thesis does not contain data or information acquired in an illegal way,*
- niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów lub tytułów zawodowych,
- *the diploma thesis has never been the basis of any other official proceedings leading to the award of diplomas or professional degrees,*
- wszystkie informacje umieszczone w niniejszej pracy, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami,
- *all information included in the diploma thesis, derived from printed and electronic sources, has been documented with relevant references in the literature section,*
- znam regulacje prawne Politechniki Warszawskiej w sprawie zarządzania prawami autorskimi i prawami pokrewnymi, prawami własności przemysłowej oraz zasadami komercjalizacji.
- *I am aware of the regulations at Warsaw University of Technology on management of copyright and related rights, industrial property rights and commercialisation.*

Oświadczam, że treść pracy dyplomowej w wersji drukowanej, treść pracy dyplomowej zawartej na nośniku elektronicznym (płycie kompaktowej) oraz treść pracy dyplomowej w module APD systemu USOS są identyczne.

I certify that the content of the printed version of the diploma thesis, the content of the electronic version of the diploma thesis (on a CD) and the content of the diploma thesis in the Archive of Diploma Theses (APD module) of the USOS system are identical.

....
czytelny podpis studenta
legible signature of the student

Contents

List of Symbols and Abbreviations

GNN – Graph Neural Network
DAG – Directed Acyclic Graph
HCR – Hierarchical Correlation Reconstruction
ADR – Adverse Drug Reaction
AKI – Acute Kidney Injury
DILI – Drug-Induced Liver Injury
GAT – Graph Attention Network
SAGE – Graph SAGE
RGCN – Relational Graph Convolutional Network
AUC – Area Under the Curve
AUPRC – Area Under the Precision-Recall Curve
PyG – PyTorch Geometric

1. Introduction

Krótki opis dziedziny problemu.

Motywacja, dlaczego warto zająć się rozwiązaniem postawionego problemu.

Opis wkładu własnego - co zrobiono w ramach przygotowywania pracy dyplomowej. Np. przygotowanie zbioru danych, opracowanie i implementacja architektury modelu itp.

YOLO [1]

2. Main objective

The main objective of this thesis is to demonstrate how knowledge graphs can be used in the medical domain in combination with experimental data, i.e., real-world patient data. The thesis addresses two complementary tasks:

- **Task A: Link prediction**, which reconstructs and updates the knowledge graph on the basis of real-world patient data;
- **Task B: Patient risk prediction** (a node-classification task), which predicts clinical outcomes using the knowledge graph and real-world patient data.

They form a potential closed-loop approach to patient-centred care in a real-world medical setting. Link prediction focused more on sharing medical knowledge between different stakeholders (hospitals, clinicians, patients), while risk prediction focuses on the individual patient outcomes. As shown in Figure 2.1, these two tasks could operate as an iterative process for improving patient care.

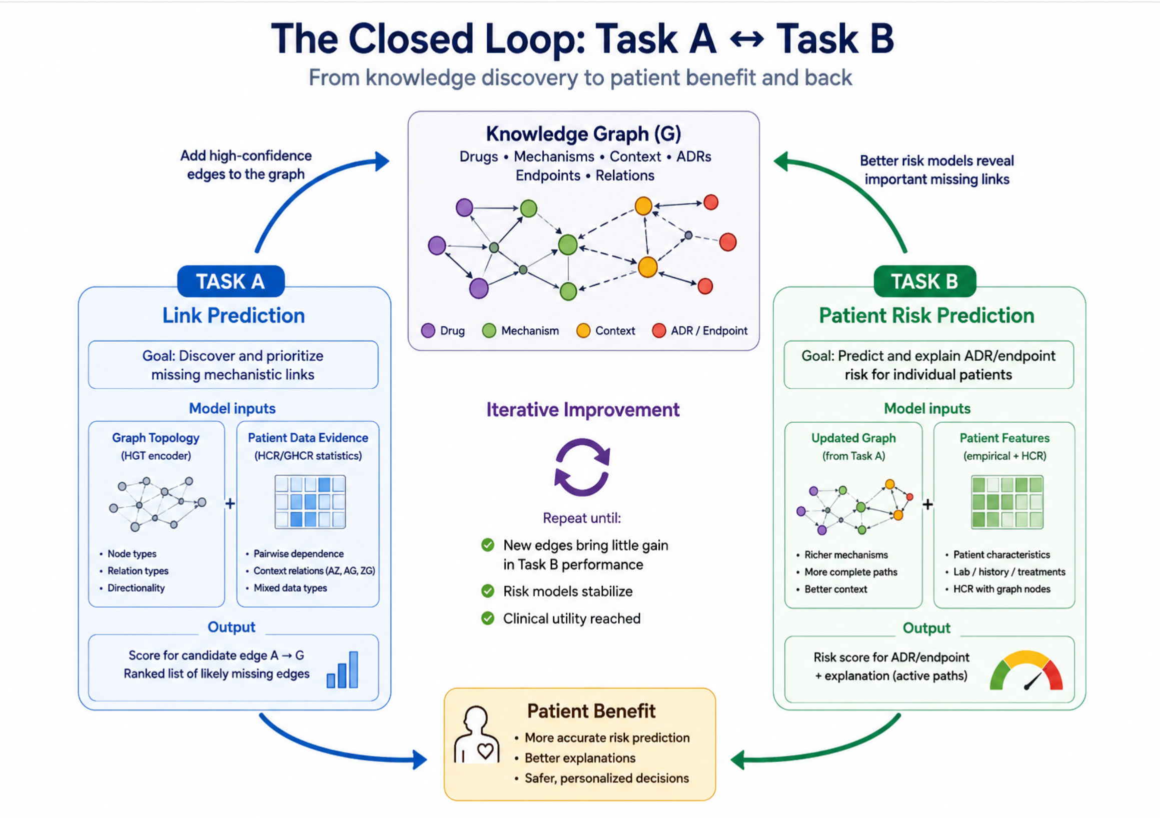

Figure 2.1. Overview of the two tasks addressed in this thesis.

In addition to this domain-oriented objective, the thesis investigates heterogeneous GNNs enhanced with a layer of dependencies calculated by Hierarchical Correlation Reconstruction (HCR). With this approach we try to examine whether GNNs performance can be improved by incorporating statistical dependencies avoiding computationally costly extensions to the model architecture.

3. Related work

In this section, we review the concepts and methods underlying both tasks addressed in this thesis: knowledge graphs as structured medical knowledge, graph neural networks for representation learning on heterogeneous graphs, and Hierarchical Correlation Reconstruction (HCR) as a statistical complement to learned graph representations. We begin with graph-based knowledge representation, then describe deep learning on graphs, and finally introduce HCR.

3.1. Knowledge graphs

Knowledge graphs represent structured knowledge through entities and the relations between them. Formally, a knowledge graph G is a set of triples

Definition 1 (Knowledge Graph)**.**

$$G \subseteq \mathcal{V} \times \mathcal{R} \times \mathcal{V},$$

where $\mathcal{V}$ denotes the set of entities (nodes) and $\mathcal{R}$ the set of relation types (edge labels). Each triple $(h, r, t) \in G$ corresponds to a directed, labeled edge $h \xrightarrow{r} t$, where h and t are referred to as the head and tail entity, respectively Hogan et al. [2].

The key benefit of knowledge graphs is that they provide an interlinked representation of knowledge in a human- and machine-understandable format (Barrasa et al. [3]). In biomedical applications, entities may represent drugs, diseases, genes, proteins, mechanisms, patient characteristics, or clinical outcomes, while edges describe typed relationships between them. Such graphs are naturally heterogeneous, because both nodes and relations can carry different semantic types. This motivates the heterogeneous-graph formulation:

Definition 2 (Heterogeneous Graph)**.** *A heterogeneous graph extends Definition 1 with explicit type information and is defined as a quadruple*

$$G = (V, E, \tau, \phi),$$

where V is the set of nodes, $E \subseteq V \times V$ is the set of edges, and $\tau : V \to \mathcal{T}_V$, $\phi : E \to \mathcal{T}_E$ are type-mapping functions assigning each node and edge a type from finite type sets $\mathcal{T}_V$ and $\mathcal{T}_E$ [4]. For an edge $e = (u, v) \in E$, the triple $\big(\tau(u), \phi(e), \tau(v)\big)$ defines its schema-level relation, which corresponds directly to the `edge_type` representation used in the `HeteroData` object in this thesis.

Knowledge graphs have been widely applied in biomedical research, particularly for integrating information from multiple sources and supporting drug discovery (MacLean [5]). However, many biomedical knowledge graphs primarily encode *associations* extracted from literature and curated databases, not patient-specific empirical realizations of the mechanisms they describe. Popular graph-based methods therefore often “highlight biological associations, failing to model causal relationships in complex dynamic biological systems” [5]. Recent work has increasingly considered integrating knowledge graphs with causal or mechanistic representations [6], [7], [8].

Beyond biomedicine, knowledge graphs are also combined with empirical interaction data in recommendation systems, where models such as KGAT propagate representations over a collaborative knowledge graph built from user–item interactions and external knowledge [9]. The Last-FM benchmark from this line of work is used as a cross-domain proxy in link prediction task. The reliability of this benchmark was subsequently questioned by Shehzad et al. [10] in a reproducibility study of intent-aware recommendation models; the corrected Last-FM* split used in this thesis is described in Section 7.

3.2. Graph Neural Networks

Graph neural networks (GNNs) learn node representations by aggregating information from neighbouring nodes through message passing [11]. After K message-passing layers, the representation of a node can incorporate information from its K-hop neighbourhood. During training, every node is represented not only by its own features, as in traditional neural networks, but also by the features of its neighbours. Unlike earlier graph-embedding methods with fixed representations, GNNs build node embeddings inductively through aggregation, so inference does not require all nodes to be present during training.

Formally, the representation $h_v^{(K)}$ of node v depends on the features of all nodes within its K-hop neighbourhood. At layer k,

$$h_v^{(k)} = \mathrm{UPDATE}\Big(h_v^{(k-1)},\ \mathrm{AGGREGATE}\big(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\}\big)\Big), \tag{1}$$

so that information from a node u at graph distance d from v only reaches v once $K \geq d$. Different GNN architectures mainly differ in how neighbouring information and, on heterogeneous graphs, relation types are transformed and aggregated. Several architectures relevant to this thesis are reviewed below.

3.2.1. GraphSAGE

GraphSAGE [11] was introduced for inductive and scalable representation learning on large graphs. It instantiates the general message-passing scheme using one of three aggregation functions—mean, LSTM, or pooling—so that the same learned functions can later be applied to unseen nodes. GraphSAGE concatenates a node’s own representation with the aggregated neighbour representation and L2-normalises the result after each layer. To improve scalability, it samples a fixed number of neighbours per node rather than using the full neighbourhood.

3.2.2. Relational Graph Convolutional Network

Relational Graph Convolutional Networks (R-GCNs) [12] extend graph convolutional networks to multi-relational graphs, in which edges represent different types of relationships. Unlike a standard GCN, which applies the same transformation to messages from all neighbours, R-GCN applies a relation-specific transformation.

For a node v, the representation at layer $l+1$ is computed as

$$h_v^{(l+1)} = \sigma\left(\sum_{r\in\mathcal{R}}\sum_{u\in\mathcal{N}_r(v)} \frac{1}{c_{v,r}} W_r^{(l)} h_u^{(l)} + W_0^{(l)} h_v^{(l)}\right), \tag{2}$$

where $\mathcal{R}$ is the set of relation types, $\mathcal{N}_r(v)$ denotes the set of neighbours connected to node v through relation r, $W_r^{(l)}$ is a trainable weight matrix specific to relation r, and $c_{v,r}$ is a normalisation constant. The term $W_0^{(l)} h_v^{(l)}$ is a self-loop transformation that preserves the node's own representation. Finally, σ denotes a non-linear activation function.

When the number of relation types is large, assigning a separate full weight matrix to each relation can lead to overfitting. R-GCN therefore introduces basis decomposition, in which each relation-specific matrix is expressed as a linear combination of B shared basis matrices:

$$W_r^{(l)} = \sum_{b=1}^{B} a_{rb}^{(l)} V_b^{(l)}, \tag{3}$$

where $V_b^{(l)} \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$ is the b-th shared basis matrix and $a_{rb}^{(l)}$ is a trainable coefficient specifying the contribution of this basis to relation r.

3.2.3. Transformer-based graph convolutions

Attention-based GNNs learn the importance of neighbouring nodes rather than using a fixed aggregation rule [13]. Transformer-based graph neural networks adapt the self-attention mechanism from the Transformer architecture Vaswani et al. [14] to graph-structured data. For each node i, its input representation $\mathbf{h}_i$ is projected into query, key, and value vectors:

$$\mathbf{q}_i = \mathbf{W}_Q\mathbf{h}_i, \qquad \mathbf{k}_i = \mathbf{W}_K\mathbf{h}_i, \qquad \mathbf{v}_i = \mathbf{W}_V\mathbf{h}_i, \tag{4}$$

where $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ are learnable weight matrices. For an edge from source node j to target node i, the attention coefficient is

$$e_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}}, \tag{5}$$

where d_k denotes the dimensionality of the key vectors.

The attention coefficients are normalised across all neighbours $\mathcal{N}(i)$ of the target node using the softmax function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{r \in \mathcal{N}(i)} \exp(e_{ir})}. \tag{6}$$

The node's representation is updated by aggregating value projections of its neighbours, weighted by the learned attention coefficients:

$$\mathbf{h}_i' = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{v}_j. \tag{7}$$

To capture multiple complementary interaction patterns, graph Transformers commonly use multi-head attention. For each head $m \in \{1, \ldots, H\}$, independent projection matrices $\mathbf{W}_Q^{(m)}$, $\mathbf{W}_K^{(m)}$, and $\mathbf{W}_V^{(m)}$ are learned. The output of head m for node i is:

$$\mathbf{h}_i'^{(m)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(m)} \mathbf{v}_j^{(m)}. \tag{8}$$

The outputs of all H heads are combined by concatenation or averaging.

All architectures mentioned in this section can be applied to heterogeneous graphs through an appropriate configuration of the `HeteroConv` layer in PyTorch Geometric (PyG). `HeteroConv` is a generic wrapper that applies a separately specified message-passing operator to each edge type, i.e., each meta-relation between a source node type and a target node type Wójcik [15].

For attention-based message passing on heterogeneous graphs, the Heterogeneous Graph Transformer (HGT) makes attention scores and message transformations dependent on the types of source and target nodes and on the edge relation type Hu et al. [4]. For an edge from source node s to target node t, HGT represents its semantics as a meta-relation $(\tau_s, \phi(e), \tau_t)$, where τ_s and τ_t denote the source and target node types and $\phi(e)$ denotes the edge type. The target node produces a query, whereas the source node produces a key and a value. Relation-specific transformations affect both the attention score and the transferred message. HGT can therefore assign different importance to neighbours depending not only on their features, but also on the semantic type of their connection to the target node.

3.3. HCR

Endpoint prediction is a node classification tasks in data.

4. Dataset

Deep learning research on medical knowledge graphs currently leaves two related gaps unaddressed. First, causal modeling is rarely applied at the level of the full patient trajectory. Second, most knowledge graphs encode relationships extracted from literature, without integrating patient-level empirical data on how outcomes are actually observed (as well noted by Toonsi et al. [6]). To address both gaps, we generated a synthetic dataset that jointly encodes a known causal structure and its patient-level empirical realizations, providing ground truth for both link prediction and node classification. This gave a full control over how knowledge is encoded in the graph, avoiding reconstruction from a partially observed real-world dataset. The generation process, the resulting graph, and the derived patient data are described in detail in this chapter.

Additionally, there is included a description of benchmark datasets used for testing the prototyped models.

4.1. Synthethic dataset

The dataset includes two structures:

1. a causal knowledge graph (DAG) that encodes causal patient pathways from patient characteristics and drug exposures to clinical events (endpoints);
2. an empirical dataset — a patient-level table with each patient's characteristics and observations, generated from the knowledge graph (Section 4.1.2).

4.1.1. Knowledge graph

The construction of the knowledge graph started from defining the clinical endpoints to model and predict, and identifying which patient characteristics and drug exposures are known to influence them. This approach – starting from the target and working backwards to its determinants – was chosen so that every path in the graph corresponds to a patient clinical pathway.

As our team includes a team member with a pharmacology background, her domain knowledge was used for constructing and verifying the initial version of the knowledge graph – the first set of endpoints, risk factors, and drug classes. Candidate mechanistic pathways connecting a drug exposure to a given endpoint, including specific drug-drug and drug-disease interaction gates, were proposed with the support of Fable[1]. All proposals generated with its support were accepted, rejected, or modified by the team member with a pharmacology background before being incorporated into the graph specification.

The graph was built iteratively. An initial, smaller version of the DAG covered a limited set of drug classes, mechanisms, and endpoints. Once this core structure was validated, it was progressively extended with additional nodes and edges. The final version includes

[1] Fable refers to Claude Fable 5, a large language model developed by Anthropic, which was used as an AI-assisted modelling aid to draft candidate mechanistic pathways for review.

155 nodes organised into six layers specified in Tables 4.1 and 405 directed edges grouped into categories in 4.2.

Table 4.1. Node types in the patient causal DAG.

Node type	Description
`patient_context`	Patient-level covariates (e.g. demographics, comorbidities).
`drug_exposure`	Drugs the patient is exposed to.
`mechanism`	Intermediate physiological mechanisms mediating the effect of a drug or context on further stages.
`adr_or_intermediate_state`	Intermediate adverse or physiological states positioned between mechanisms and clinical endpoints.
`observation_or_selection`	Nodes representing observation, documentation, or selection processes rather.
`clinical_endpoint`	Target adverse clinical outcomes (e.g. AKI, DILI, Hospitalization).

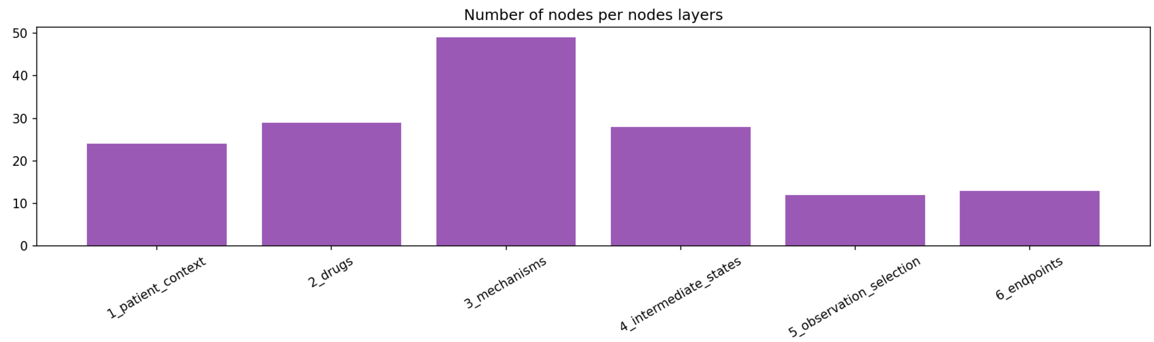

Figure 4.1. Frequency of nodes types.

Beyond its topology, every node in the graph carries a set of attributes which are used directly in the data-generating process described in Section 4.1.2:

- `base_prevalence` encodes a population-level probability of a node being present in a patient who has none of the node's risk factors active.
- `severity_weight` encodes the clinical severity assigned to a node, independent of how frequently it occurs.
- `observability` encodes how reliably a true clinical state is expected to be recorded in practice.

As edges bring different information based on edge type, they were assigned declared attributes, such as:

As edges bring different information based on edge type, they were assigned declared attributes in the audited graph specification:

Table 4.2. Semantic categories of directed edges in the patient causal DAG.

Category	**Included edge types**
Background risk and treatment assignment	`risk_modifier`, `confounding`, `latent_confounding`, `treatment_indication`, `treatment_burden`
Drug effects and mechanistic pathways	`drug_to_mechanism`, `causal_mechanistic`, `mechanism_to_adr`, `adr_to_intermediate`
Clinical outcome progression	`adr_to_endpoint`, `endpoint_to_hospitalization`
Observation and selection processes	`observation`, `selection`, `observation_process`
Polypharmacy, aggregate burden, and interactions	`drug_to_burden`, `drug_to_load`, `interaction_gate_component`, `drug_pairwise_gate`, `drug_triple_gate`, `interaction_amplify`, `adr_burden_component`, `burden_threshold`

- `effect_size` encodes the strength of the causal effect of the source node on the target node.
- `effect_sign` encodes the direction of that effect (risk-increasing or risk-decreasing).
- `activation_frequency` encodes how often the source variable is active or clinically relevant in the population.
- `mechanistic_confidence` encodes the confidence assigned during graph audit that the mechanistic link is plausible.
- `evidence_weight` encodes the weight of supporting evidence for the edge in the audited knowledge graph.

In fig. 4.2 it is visualized an overview of DAG graph.

The final version of the DAG was verified statistically with the metrics reported in table 4.3. They confirm the graph's acyclicity and characterize its structural complexity for modelling purposes.

4.1.2. Patient data

The patient-level data generation procedure was designed to produce tabular records that provide empirical evidence for knowledge-graph reconstruction and downstream prediction of clinical outcomes. Its purpose was to transform the static causal DAG specification into a patient-level dataset in which each individual is represented by a realised configuration of exposures, clinical states, and outcomes. This enables the graph representation to be used as a dynamic, data-supported structure for multiple downstream tasks. For each of 20,000 simulated patients, the generator assigns a realised value to each of the 155 nodes in the causal DAG. Values are generated in topological order, following the

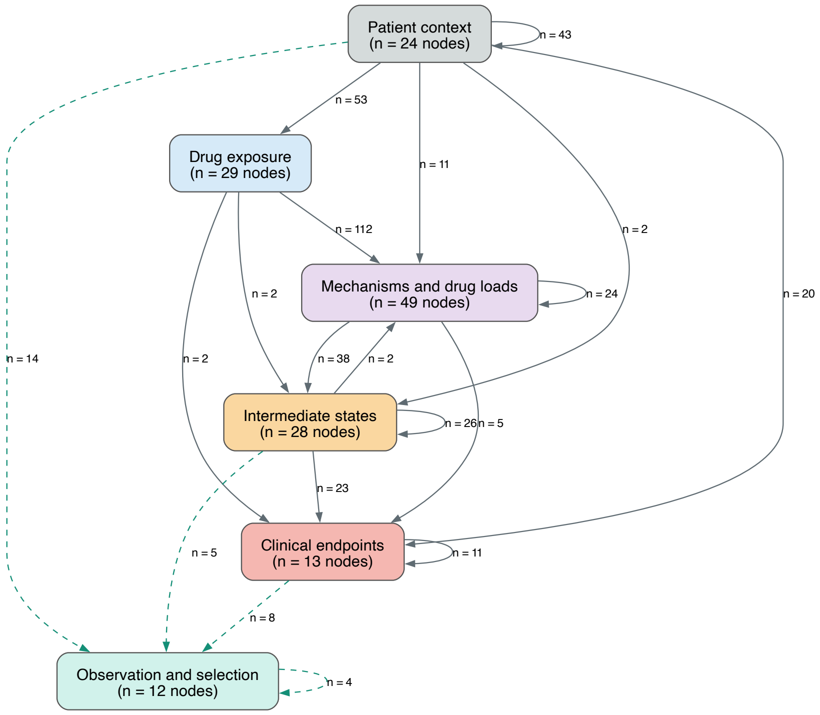

Figure 4.2. DAG visualization

causal direction of the graph. For every patient, upstream context variables are generated first, followed by drug exposures, physiological mechanisms, and intermediate adverse states. Clinical endpoints are generated only after all their parent variables have been realised. Each node value is generated by one of mechanisms: a stochastic process, or a deterministic computation of other nodes. The following paragraphs describe each mechanism.

Stochastic nodes Most nodes – patient covariates, drug exposures, intermediate mechanisms, and clinical endpoints – are generated as Bernoulli draws whose probability is a logistic function of their parents:

$$\eta_i = \mathrm{logit}(p_i^0) + \sum_{j \in \mathrm{parents}(i)} s_{ij} \cdot w_{ij} \cdot x_j, \qquad x_i \sim \mathrm{Bernoulli}\big(\sigma(\eta_i)\big), \tag{9}$$

where p_i^0 is the node's baseline prevalence, x_j is the realised value of parent j, and w_{ij}, s_{ij} are the effect size and effect sign declared on the edge $j \to i$. Nodes are generated in topological order, so that every parent has already been assigned a value before its children are processed.

Three conditions affect the implementation:

- A parent with more than two distinct realised values (i.e. a `count` or `continuous`

Table 4.3. Structural statistics of the causal knowledge graph.

Metric	Value
Nodes / edges	155 / 405
Graph density	0.017
Acyclicity	True
Self-loops / duplicate edges	0 / 0
Source nodes / sink nodes	9 / 12
Direct parents per clinical endpoint	mean 4.7 (range 1–14)
Ancestors per clinical endpoint	median 31 (range 9–126)
Longest root-to-endpoint path	median 12 edges (range 5–16)
Shortest root-to-endpoint path	1 edge for 10 of 13 endpoints; 3–4 edges for the three rarest endpoints
Drug–endpoint pairs sharing a common ancestor	85

node, such as `polypharmacy_burden`) is standardised before entering the sum, $(x_j - \bar{x}_j)/\mathrm{sd}(x_j)$, so that its contribution to η_i is on a comparable scale to binary parents.

- p_i^0 is taken from the `base_prevalence` column of the node specification [2].
- If missing, `effect_size`, and `effect_sign` defaults to 0.5 and +1 accordingly.

Deterministic derived variables. Some variables are derived deterministically from values that have already been generated based on patterns of polypharmacy, drug interactions, and adverse-event burden explicit in the graph.

Table 4.4 summarises the deterministic variables and their generating rules.

A small subset of deterministic variables represents recorded clinical evidence. Such variables are computed as the product of an underlying state and its associated observation process. For example,

$$\texttt{elevated\_creatinine\_recorded} = \texttt{creatinine\_rise} \times \texttt{creatinine\_tested}.$$

Thus, a creatinine rise may be present but remain absent from the recorded data if no creatinine test is performed. Analogous deterministic rules are applied to recorded ALT/AST elevation and recorded QT prolongation.

The observation variables distinguish clinical truth from its documentated representation. They provide a structured representation of testing and recording processes that are relevant to the construction and interpretation of alternative observational scenarios.

[2] If base prevalence value is missing, a type-dependent default is used instead (0.16 for drug exposures, 0.02 for clinical endpoints, 0.35 for observation/selection nodes, 0.10 otherwise)

Table 4.4. Deterministic variables in the synthetic patient generator.

Variable group	Rule type	Generating rule
Renal, CNS, bleeding, serotonergic, and hepatic ddi_* gates	Conjunctive gate	A binary variable equal to 1 only when all listed drug-exposure components are active. For example, ddi_renal_triple_whammy = nsaid $\wedge$ acei_arb $\wedge$ diuretic.
Drug–disease interaction gates (drug_disease_*)	Conjunctive gate	A binary variable equal to 1 when the relevant drug exposure or drug load and the associated patient risk factor are both active.
active_drug_count	Drug count	Sum of all 29 binary drug_exposure variables.
Drug-load variables (nephrotoxin_load, hepatic_drug_load, qt_drug_load_v3, cns_depressant_load, bleeding_risk_load, serotonergic_load, cyp_inhibitor_load)	Drug count	Sum of active drug exposures belonging to a predefined clinical risk group.
ddi_qt_multidrug_load	Drug count	Sum of active QT-prolonging drug exposures specified in the generator.
ddi_serotonergic_high_load	Threshold	Equal to 1 when serotonergic_load is at least 2.
cumulative_adr_burden	Weighted sum	Severity-weighted sum of intermediate adverse states plus 0.08 × active_drug_count.
severe_adr_burden	Threshold	Equal to 1 when cumulative_adr_burden is at least 4.0.

4.1.3. Evaluation scenarios

Data was generated in five controlled scenarios constructed to evaluate model robustness under common challenges of observational clinical data. All scenarios use the same underlying synthetic pharmacotherapy DAG and preserve the semantic meaning of node and edge types. Each scenario modifies either the available information, the sampling mechanism, the treatment-assignment process, the observation process, or the institutional context.

For example, the `selection_bias` scenario restricts the cohort to patients with hospital contact, creatinine testing, and liver-function testing. Consequently, inclusion depends on downstream healthcare-contact and diagnostic processes. The observation and documentation variables therefore support interpretation of this scenario.

All of scenarios are presented in Table 4.5.

Table 4.5. Synthetic evaluation scenarios and their intended data challenge.

Scenario	Primary purpose
`clean`	Reference setting with complete generated information.
`hidden_confounder`	Unmeasured confounding: `unobserved_severity` remains active in the generator but is withheld from the observed data.
`selection_bias`	Selection bias induced by restricting the cohort to patients with hospital contact, creatinine testing, and liver-function testing.
`no_overlap`	Changed treatment-assignment probabilities: selected drug exposures become rare within specific clinical subgroups.
`noisy_documentation`	Imperfect clinical recording, including false negatives and a 1% false-positive recording probability.
`multihospital`	Institutional heterogeneity in treatment assignment and documentation processes across four synthetic hospitals.

4.1.4. Resulting patient-level dataset

Th final dataset generated contains 20,000 simulated patients for every scenario described. Each row represents one patient and each graph node is represented by a corresponding patient-level variable. Table 4.6 summarises the structure and value types of the resulting dataset. A complete variable-level data dictionary is provided in Appendix 8.

Clinical endpoints The 13 clinical endpoints are binary stochastic variables generated after their direct parents have been realised. Their values are the ground-truth labels used for endpoint prediction. Table 4.7 lists the endpoint names, their direct parents in the final DAG, and the generator-based oracle AUC on the complete synthetic cohort (each patient scored with the known generative logistic model; relative performance is reported later via eq. (12)).

Table 4.6. Schema of the generated patient-level dataset.

Variable group	Number	Value type
`patient_id`	1	Integer identifier.
`hospital_id`	1	Categorical integer identifier.
`patient_context`	24	Mostly binary; selected count and baseline-measurement variables.
`drug_exposure`	29	Binary.
`mechanism`	49	Binary, count, and deterministic gate variables.
`adr_or_intermediate_state`	28	Mostly binary; derived score and threshold variables.
`observation_or_selection`	12	Binary.
`clinical_endpoint`	13	Binary.

Table 4.7. Endpoint prevalence and generator-based oracle AUC in the complete synthetic cohort.

Endpoint	Positive cases	Prevalence (%)	Oracle AUC (%)	Parents
AKI	2,620	13.10	81.20	6
DILI	361	1.81	69.30	7
Delirium	739	3.70	65.73	4
Depression	1,693	8.47	66.27	6
Falls	1,558	7.79	68.09	7
GI bleeding	425	2.13	61.28	5
Hospitalization	4,502	22.51	76.23	14
Hyperkalemia	969	4.85	59.63	2
Hyponatremia	1,574	7.87	62.69	3
QT arrhythmia	319	1.60	61.66	4
Serotonin syndrome	51	0.26	52.16	1
Rhabdomyolysis	69	0.35	58.64	1
Lactic acidosis	111	0.56	52.45	1

Patient-level data splits Patients were partitioned into training, validation, and test sets in proportions of 70%, 15%, and 15%, respectively. For the `clean` scenario containing 20,000 patients, this corresponds to 14,000 training patients, 3,000 validation patients, and 3,000 test patients.

Splits were created at the `patient_id` level using a deterministic greedy multi-label balancing procedure with seed `20260722`. Patients with rare endpoint combinations were assigned first, and assignments were chosen to preserve the prevalence of each clinical endpoint across the three splits. Consequently, every patient appears in exactly one split, while the multi-label endpoint distribution remains approximately balanced.

A single master split assignment was derived from the `clean` scenario and reused across all alternative scenarios. This ensures that performance differences between scenarios reflect the intended changes in data scenarios.

4.2. Benchmark Dataset

4.2.1. Last-FM dataset

4.2.2. Hetionet knowledge graph

Hetionet is a heterogeneous network (*hetnet*) that integrates biomedical knowledge from 29 publicly available resources. It represents biological and pharmacological entities as nodes and documented relationships between them as typed edges.

Hetionet v1.0 contains 47,031 nodes belonging to 11 node types and 2,250,197 relationships of 24 types. The graph includes compounds, diseases, genes, anatomies, pathways, biological processes, molecular functions, cellular components, pharmacologic classes, side effects, and symptoms. This heterogeneous structure makes Hetionet suitable for node classification task described further. The details of benchmark check are included in Section 7.2.

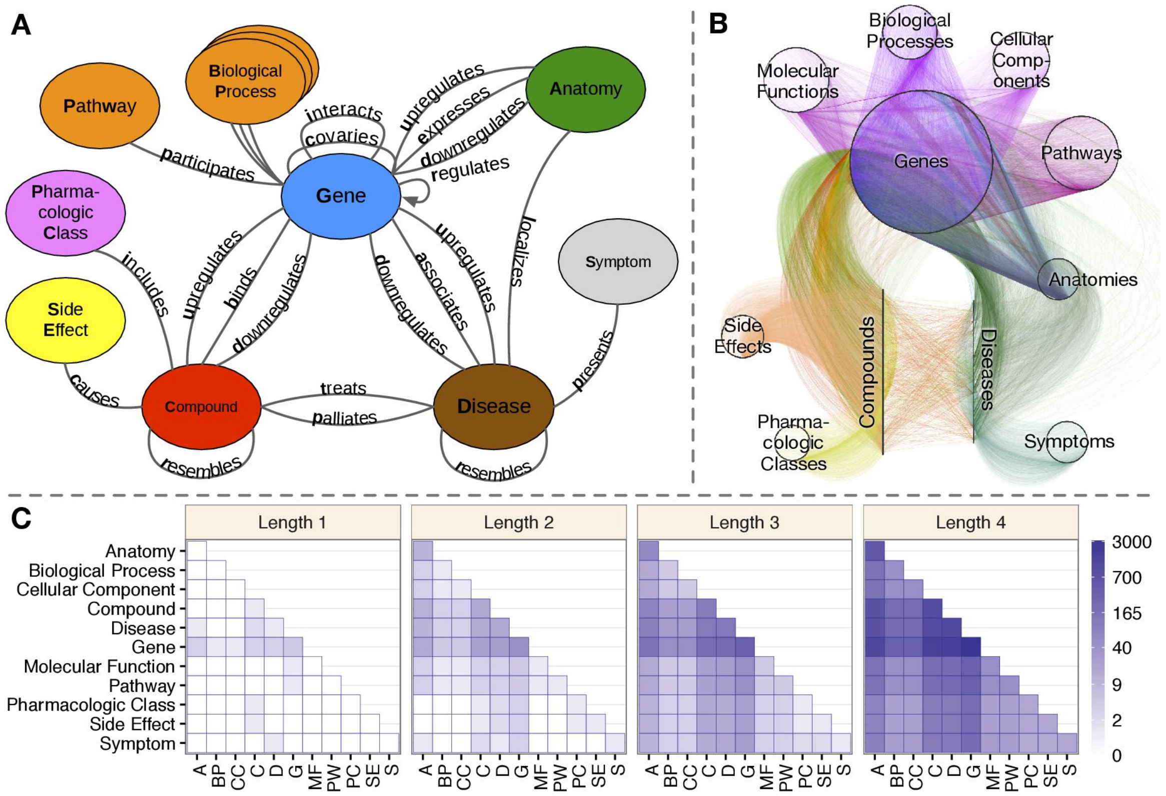

Figure 4.3. Hetionet knowledge graph

5. Main Framework

A causal knowledge graph is not merely a static record of the literature: it can be systematically audited, weighted, and enriched with observational data, and subsequently used as the basis for explainable clinical prediction of drug-related outcomes. This thesis operationalizes that idea through two complementary graph neural network tasks, applied to a shared causal knowledge graph of pharmacotherapy:

a) **Task A:** link prediction, which reconstructs and updates the knowledge graph based on real patient data;

b) **Task B:** node classification, which predicts patient-specific clinical endpoints based on the knowledge graph and real patient data.

Since both tasks operate on the same underlying knowledge graph, we adopt graph neural networks as the common modeling framework, and enrich them with Hierarchical Correlation Reconstruction (HCR). In Figure 5.1 there is presented a high-level approach to construct GNN with HCR. While the two tasks share this common foundation, their implementation details differ in several respects; we therefore present each task separately in the following subsections.

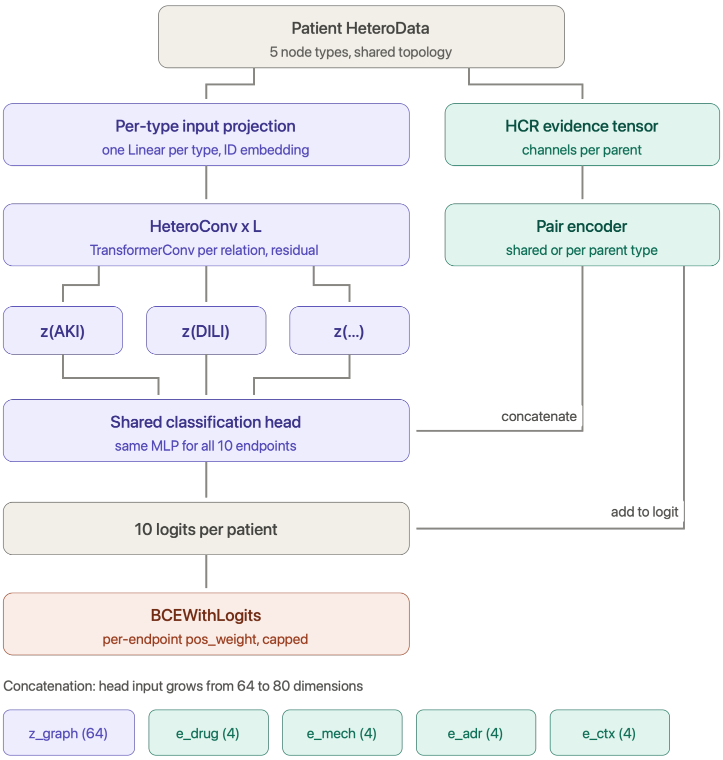

Figure 5.1. The presentation of approach to node classification task.

Table 5.1. Components of the per-patient `HeteroData` graph representation.

Component	**Description**
Node types	Six semantic categories of nodes shared across all patient graphs: `patient_context`, `drug_exposure`, `mechanism`, `adr_or_intermediate_state`, `observation_or_selection`, and `clinical_endpoint`.
Relations	Keyed by `(src_type, "to", dst_type)`; `edge_type` is encoded as a one-hot block inside `edge_attr`. This keeps the number of distinct convolutional relation modules in `HeteroConv` small while still letting edge-type information reach the message function.
Edge attributes (`edge_attr`)	Fixed-length vector per edge (see table 5.2): signed effect `effect_size×effect_sign`, audit scalars `activation_frequency`, `mechanistic_confidence`, `evidence_weight`, and a one-hot `edge_type` block.
Node features (`x`)	The patient's observed values for a node

5.1. Link prediction in Graph Reconstruction

5.2. Endpoint prediction

Task B is formulated as multi-label node classification on a heterogeneous, per-patient graph. Instead of building one global graph and treating patients as additional nodes, every patient is represented as a copy of the same fixed causal DAG topology, with node feature vectors instantiated from that patient's individual covariates. Formally, for a patient graph $G_p = (V, E)$ shared across the cohort, with patient-specific node features x_p. $\mathcal{V}_{\text{target}} \subset V$ denote the set of endpoint nodes used as prediction targets. The model computes

$$\hat{y}_{p,v} = \sigma\big(f_\theta(G_p, x_p)_v\big), \quad v \in \mathcal{V}_{\text{target}}$$

where f_θ is a heterogeneous graph neural network shared across all patients, and only the endpoints target embeddings are read out by the classification head.

5.2.1. Data Representation

The first thing in applying GNNs is to built properly the input into neural network. The input graph for the node classification task was built based on:

a) **nodes**
b) **edges**
c) **per-scenario patient sample table**

Based on those items `HeteroData` object was created which was represented by objects type presented in the table

The patient data is encoded on nodes level. In fig. 5.1 there is presented the exemplary DAG for one of patient.

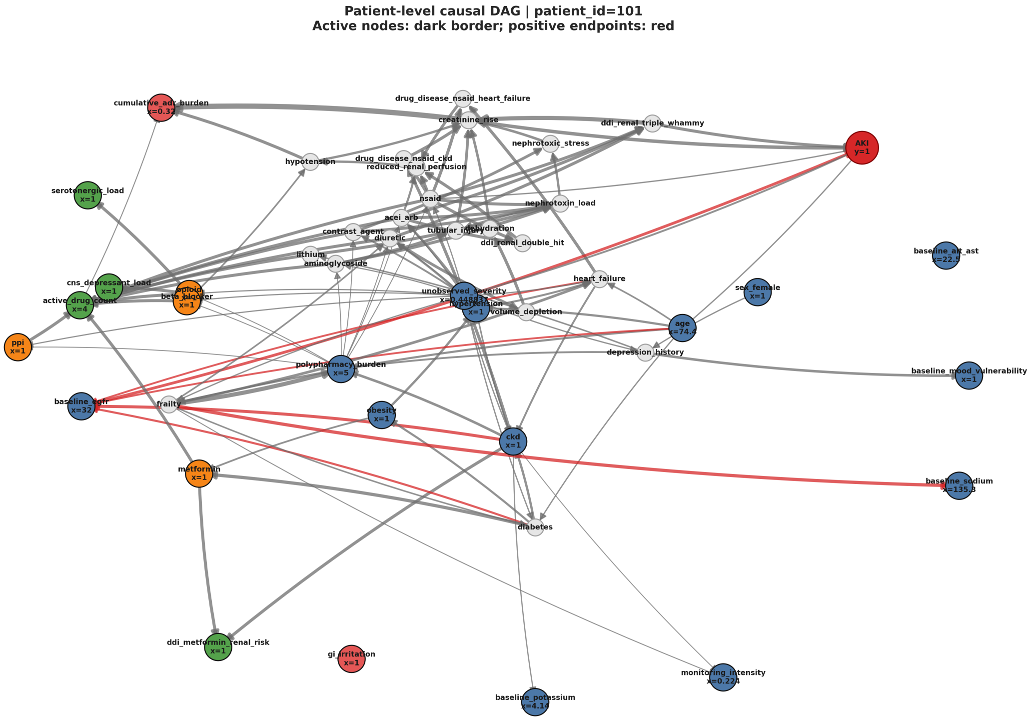

Figure 5.2. The description of the two tasks addressed in the thesis.

Two structural exclusion mechanisms are applied before any feature is computed:

a) nodes flagged `exclude_from_observed_model` or `is_latent` (hidden confounders);
b) nodes that are structural descendants of any endpoint in the DAG (e.g. documentation nodes such as `hospital_contact`, `fall_reported`, which are caused by the endpoint rather than causal parents of it)

All normalization statistics (mean/std for continuous node values) and all statistical evidence features (section 5.2.3) are computed exclusively on the training split of patients, to prevent both node-level and patient-level information leakage across splits.

Prediction targets. Although the knowledge graph defines 13 clinical outcomes nodes, the model is trained and evaluated on a subset of 11 clinical endpoints, excluding `Hospitalization` and `Falls`. They are retained in the per-patient graph topology and in patient labels, but are excluded from the multi-target prediction head.

This choice follows both the graph structure and the clinical scope of the task. `Hospitalization` was excluded because the DAG models it as a downstream consequence of other endpoints (`endpoint_to_hospitalization`), which makes it a poor fit for parallel multi-target prediction alongside those same outcomes. `Falls`, by contrast, is

Table 5.2. Layout of the Task B `edge_attr` vector.

Block	Columns	Source
Signed effect	1	`effect_size` × `effect_sign`
Audit scalars	3	`activation_frequency`, `mechanistic_confidence`, `evidence_weight`
Edge-type one-hot	|`edge_type`|	semantic edge category from the audited DAG

Table 5.3. Use of `edge_attr` by convolution backbone in Task B.

Backbone	Edge features used in message passing
GraphSAGE	none (topology only; `edge_attr` stored but not passed to `SAGEConv`)
TransformerConv	full `edge_attr` via `edge_dim`
R-GCN	relation identity via separate modules per edge type, not via the Task B `edge_attr` vector

a geriatric outcome that has different casual pathways than other endpoints that originate from exposures and mechanistic intermediates in the pharmacotherapy graph. Based on these conditions, falls and hospitalisation are treated as downstream, multifactorial events outside that primary prediction scope.

For reporting, all eleven targets are trained, but macro-averaged metrics (AUC, AP, % of oracle AUC; eq. (12)) are computed on eight endpoints only. `Serotonin_syndrome`, `Rhabdomyolysis`, and `Lactic_acidosis` are omitted from the aggregate because their extreme rare prevalence makes split-level AUC and AP unstable.

5.2.2. Model architecture

The core architecture is a stack of heterogeneous graph convolutional layers wrapped in `HeteroConv`, shared across relation types but parameterized identically for every patient graph. The following convolution backbones are supported through a common registry and were benchmarked against each other: GraphSAGE, R-GCN, and TransformerConv. All three are applied to the heterogeneous patient DAG via `HeteroConv`. Each directed edge in the graph carries a fixed `edge_attr` vector assembled at graph-build phase from the audited edge table (table 5.2). In model design relations are grouped by node-type pairs `(src_type, to, dst_type)` and the semantic `edge_type` is retained as a one-hot suffix in `edge_attr`. This vector is consumed differently depending on the convolution operator (table 5.3).

A design issue that was handled during experiments is the duplication of the self-transform across relations. Several PyG convolution layers (SAGEConv, TransformerConv) internally add their own linear self-loop term, wrapped per-relation inside `HeteroConv`, this term is silently duplicated once per incoming relation type for a given node type. The implementation removes this internal duplication (`root_weight`=False for SAGE/Transformer) and replaces it with a single, explicit, controlled self-transform per layer per node type.

A node-identity embedding, added after the input projection, was introduced to resolve a specific representational collapse. Message-passing GNNs are bounded in expressive power by the 1-dimensional Weisfeiler-Lehman graph isomorphism test. In the dataset, several endpoint nodes of the same type (e.g. Serotonin syndrome, Rhabdomyolysis, Lactic acidosis) share identical static audited attributes and comparable topological positions, and were therefore indistinguishable to the model except through such positional cues. To break this symmetry, a learnable embedding table is maintained per node type, indexed by each node's fixed local index, and added to the projected input features before the first convolutional layer, following the same principle underlying position aware node embeddings in .

This connects directly to the model's handling of oversmoothing. Using multiple message-passing layers, repeated aggregation causes node representations to converge toward indistinguishability. To mitigate this, several oversmoothing countermeasures were tested like residual connections, Jumping Knowledge, DropEdge, and PairNorm, from which only residual connections were found the most effective and were used in the final results presented in the thesis.

5.2.3. Statistical evidence branch

In addition to the deep graph representation learned by the GNN, the endpoint prediction approach includes a *statistical evidence branch*—a wide side-channel that supplies cohort-level dependence statistics. The branch is an *HCR-inspired* way of representing dependencies (section 3.3). The HCR construction—a cohort coefficient multiplied by a patient-specific activation and summed over parents—is retained as one implementation mode (*linear*). The remaining configurations extend this idea with additional contingency-table statistics, non-binary parent handling, higher-order path moments, and learned fusion with the GNN head.

This channel is motivated by the observation that some parent-endpoint dependencies are well captured by simple, well-understood pairwise statistics computed directly from the training cohort, and that combining such statistics with the deep GNN representation captures the strong signal from direct parents and smoothed signal from further layers. All modes of the statistical branch are summarized in Table 5.4 and below they is included detailed explanation of every mode characteristics.

Scope of variables and parents. For each target endpoint, evidence is computed over its *direct parents* in the patient graph (subject to the structural exclusions described in section 5.2.1). Pairwise statistics relate one parent to one endpoint at a single hop. Higher-order evidence over *grandparent–parent–endpoint* paths ($G \to P \to Y$, where G, P, and Y denote the grandparent, parent, and endpoint *nodes* on a DAG-consistent triple) is computed separately in the *triple* extension. It is stored in its own evidence tensor (13 channels per triple slot), and fused through a dedicated encoder block. Several evidence dimensionalities were evaluated, including binary-only parents (compact 10-channel evidence) and all parent value types (binary, count, and continuous; 42-channel evidence).

HCR core coefficients. For a binary parent R and binary endpoint node Y, the primary association coefficient is the φ-coefficient derived from the training-cohort contingency table. For each patient p, the parent activation is the standardized binary score $\phi_1(R)$. The HCR patient evidence for that parent–endpoint pair is the product $s_{p,rv} = a_{11,rv}\phi_1(R_p)$ (channel `s_evidence`). In *linear* mode the endpoint-wide score is the sum over eligible direct parents,

$$s_{p,v} = \sum_{r\in\mathrm{Pa}(v)} a_{11,rv}\phi_1(R_p),$$

and a single learned scalar per endpoint scales this score before it is added to the GNN logit.

Extended pairwise evidence channels. Beyond the HCR core, the same contingency table yields supplementary association measures that provide complementary views of parent–endpoint dependence for a learned encoder: normalized mutual information, a tanh-scaled log odds ratio, risk difference, tanh-scaled log lift, parent and endpoint prevalence, and support (the fraction of training patients with both variables observed). These are assembled into a fixed-order evidence vector per (patient, parent, endpoint) slot. Two pairwise dimensionalities were used:

- **10 channels**: compact binary-parent representation with explicit `s_evidence`;
- **42 channels**: enriched representation including matrix-energy summaries, marginal entropies, type indicators, and the patient activation channel.

Evidence encoders and endpoint-specific fusion. In *nonlinear* and *triple* modes, a shared `HCRPairEncoder` processes each evidence slot with the same MLP (`Linear`→`ReLU`→`Linear`), mean-pools over parent or triple slots using a padding mask, and collapses the pooled embedding to a scalar logit contribution. In *nonlinear* mode only, that contribution is further scaled by a learned per-endpoint weight (`hcr_endpoint_weight`).

In *typed_concat* (and *typed+triple*), the architecture differs in two respects. First, evidence is partitioned by *parent node type*: each of the four parent categories (`drug_exposure`, `mechanism`, `adr_or_intermediate_state`, `patient_context`) is encoded by its own `HCRPairEncoder` instance applied to a separate evidence tensor (`hcr_typed_{type}_features`). These encoders are shared across endpoints; they differ only in the semantic role of the parents they read. Second, fusion is *endpoint-specific*: a learnable gate matrix `hcr_type_gate` $\in \mathbb{R}^{|\mathcal{V}_{\text{target}}|\times B}$ (with $B = 4$ parent-type blocks, or $B = 5$ when a triple block is enabled) scales each type-level embedding separately for every target endpoint before concatenation. The gated embeddings are concatenated with the GNN target node embedding and passed jointly to the classification head, rather than being added to the endpoint logit. In *typed+triple*, triple-path evidence is encoded by an additional `hcr_triple_encoder` and included as a fifth block, subject to its own per-endpoint gate column in `hcr_type_gate`.

Non-binary parents. Count- and continuous-valued direct parents are handled outside the binary φ-coefficient framework. Values are mapped to comparable pseudo-uniform scores via a train-only empirical CDF transform; count variables receive deterministic, patient-ID-seeded jitter before transformation. Non-linear dependence on continuous parents is represented through a Legendre polynomial basis. These channels extend the binary *patient activation* in the 42-dimensional branch.

Triple-path extension. In order to capture more complex dependencies, third-order moments are computed over graph-consistent triples $G \to P \to Y$. The main coefficient a_{111} is estimated from a $2 \times 2 \times 2$ contingency table over three binary nodes on each DAG-consistent path $G \to P \to Y$ (grandparent, parent, and endpoint node). It generalizes the pairwise φ-coefficient, which uses only two variables at a time. The triple vector also retains the three pairwise marginals $(a_{11}^{GP}, a_{11}^{GY}, a_{11}^{PY})$, prevalence terms, support, patient activations, and an explicit patient evidence product $a_{111}\,\phi_G\,\phi_P$. Triple evidence uses the same slot-based encoder interface as pairwise evidence.

5.2.4. Training setup

Training iterates over patient graphs batched via PyG DataLoader. The experiments were run with targeted binary cross-entropy and focal loss with class-balance-aware weighting. Weight and focal alpha were computed from the training split's endpoint prevalence, since clinical outcomes are rare and highly imbalanced. Model selection uses ReduceLROnPlateau on validation performance, and evaluation is reported separately per endpoint and in aggregate, using AUROC, AP, and prevalence-normalized lift. As the synthetic dataset generated for this project contains rare events and class imbalance, AP was chosen as a key performance metric. It focuses on the positive class and reflects the trade-off between precision and recall. Below there are included the equations for calculating metrics.

AUC is the area under the receiver operating characteristic curve (ROC-AUC) and is defined as:

$$\text{ROC-AUC} = \int_0^1 \text{TPR}(\text{FPR})\, d\text{FPR}. \tag{10}$$

where TPR is the true positive rate and FPR is the false positive rate.

AP is the area under the precision–recall curve and is defined as:

$$\text{AP} = \sum_n (R_n - R_{n-1})\, P_n, \tag{11}$$

where P_n and R_n denote precision and recall at the n-th threshold, respectively.

Lift is the AP divided by the prevalence baseline, that is, the factor by which the model improves over a random ranking, e.g. at a mean prevalence of 5.5% a lift of 2.3 corresponds to an AP of 0.125.

In addition to the traditional metrics described above, a *generator-based oracle AUC* was computed for the node-classification task. As the dataset is synthetic, the logistic generative model for each endpoint is known: each patient is scored with the true parent effects from the audited DAG and ROC-AUC is evaluated on the resulting probabilities. Model performance is reported relative to this reference via *% of oracle AUC*:

$$\frac{\mathrm{AUC} - 0.5}{\mathrm{AUC}_{\mathrm{Oracle}} - 0.5} \cdot 100\%, \tag{12}$$

where $\mathrm{AUC}_{\mathrm{Oracle}} = 0.651$ is the macro-averaged oracle AUC over the same eight endpoints used in aggregate reporting (table 4.7). This metric expresses the share of ranking signal recoverable under the known generator, relative to random performance at 0.5. It is *not* a Bayes-optimal error bound; it is a simulation-based performance ceiling derived from the generative model.

6. Experiments results

6.1. Link prediction in graph reconstuction

6.2. Endpoint prediction

6.2.1. Architecture

The first set of experiments established how much the choice of convolution architecture matters for this task. Three convolutions were compared: R-GCN, GraphSAGE and TransformerConv. All three are defined for homogeneous graphs, so each was wrapped in `HeteroConv`, which instantiates a separate convolution for every relation type and aggregates the results at the target node and allows to preserve the heterogenous structure.

All experiments share the same encoder structure and differ only in a small number of hyperparameters: the number of layers L, the presence of a residual connection and the HCR variant. Everything else was held fixed. The shared settings are listed in table 6.1, and the parameters specific to individual convolutions in table 6.2.

All three convolutions are wrapped in `HeteroConv`, which instantiates a separate convolution for each of the 19 relation types in the graph. The parameter counts therefore scale with the number of relations. In attention-based layers the hidden width is split across heads, so the output width stays at 64 regardless of the number of heads.

Table 6.2. Architecture-specific hyperparameters.

Hyperparameter	R-GCN	GraphSAGE	TransformerConv
Attention heads	—	—	4
Units per head	—	—	16
Output width per relation	64	64	$4 \times 16 = 64$
Basis decomposition	8 bases	—	—
Uses edge attributes	relation id only	none (stored, unused)	full `edge_attr`
Weight matrices per relation	shared through bases	2	3 (query, key, value)

The results for those experiments are reported in table 6.3. All of metrics reported are described in **??**. The epoch is selected on the validation set and test metrics are reported from that epoch. The experiments assumed to used multiple number of layers as patient path is up to 6 degrees and it is assumed that every point can have an influence in casual path. For each experiment all number of layers were calculated, but for reporting reasons in table 6.3 there are included layers 1,3 and 6 to show the trend with the increased number of layers. The convention is liked that unless the results from other layers are significant for conclusion.

Without residual connections the three architectures diverge sharply at L=6, collapse to constant predictions (AUC=0.50).

Table 6.3. Message-passing architectures without residual connections.

		Validation				Test			
Architecture	L	AUC	AP	Lift	% ceil.	AUC	AP	Lift	% ceil.
R-GCN	1	0.623	0.124	2.28	81.6	0.624	0.120	2.21	82.0
GraphSAGE	1	0.622	0.124	2.28	80.9	0.619	0.120	2.21	78.9
TransformerConv	1	0.622	0.125	2.30	81.2	0.620	0.120	2.20	79.6
R-GCN	3	0.542	0.076	1.40	28.0	0.542	0.068	1.26	27.7
GraphSAGE	3	0.549	0.079	1.45	32.2	0.549	0.072	1.32	32.8
TransformerConv	3	0.550	0.083	1.53	33.2	0.552	0.073	1.35	34.5
R-GCN	6	0.503	0.055	1.00	2.2	0.500	0.054	1.00	0.0
GraphSAGE	6	0.530	0.065	1.20	20.0	0.510	0.061	1.12	6.6
TransformerConv	6	0.527	0.065	1.20	18.0	0.500	0.054	1.00	0.0

That's why the next step was application of residual connections described in **??** to see the capabilities of models with higher number of layers. The results are presented in table 6.4.

Table 6.4. The same architectures with residual connections.

		Validation				Test			
Architecture	L	AUC	AP	Lift	% ceil.	AUC	AP	Lift	% ceil.
R-GCN	1	0.625	0.125	2.30	82.7	0.625	0.121	2.22	83.2
GraphSAGE	1	0.621	0.124	2.27	80.6	0.619	0.120	2.21	78.7
TransformerConv	1	0.623	0.126	2.31	81.3	0.619	0.120	2.21	79.0
R-GCN	3	0.630	0.133	2.44	86.0	0.627	0.125	2.29	84.2
GraphSAGE	3	0.623	0.132	2.42	81.4	0.616	0.124	2.27	77.0
TransformerConv	3	0.628	0.135	2.47	85.2	0.620	0.123	2.27	79.7
R-GCN	6	0.629	0.135	2.48	85.6	0.626	0.123	2.27	83.9
GraphSAGE	6	0.628	0.134	2.46	84.9	0.624	0.127	2.33	82.3
TransformerConv	6	0.627	0.134	2.46	84.2	0.620	0.122	2.24	79.4

With residual connections all three architectures converge to comparable performance (table 6.4): 77–86% of the oracle AUC (eq. (12)) on the test set, with differences of about 0.01 AUC. The more informative factors for model assessment are AP and Lift, which show that the model performs nearly 2.3 times better than a random baseline. Since the architectures achieve comparable performance, the choice for the remaining experiments was therefore made on structural grounds. TransformerConv is the only one of the three backbones benchmarked here that consumes the full `edge_attr` vector in message passing (see table 5.3). Since edge-level audit evidence is important for graph representation, TransformerConv was selected for all subsequent experiments with the statistical evidence branch.

6.2.2. HCR configuration

HCR was introduced to supply the model with statistical evidence about grandparent/parent–endpoint dependence that the message-passing path has to infer on its own (section 5.2.3). The configurations examined here vary along three axes, each addressing a separate question. The first is the form of the wide branch: a single learned coefficient per endpoint applied to a precomputed dependence score, against a small nonlinear encoder applied to each parent separately. The second is the scope of the evidence: binary parents only, or all node types, in which case continuous and count variables are expanded in a Legendre basis. The third is the order of the statistic: pairwise coefficients between a parent and an endpoint, or third-order moments over grandparent–parent–endpoint paths, which capture interactions that pairwise coefficients cannot represent. The relevant configurations are listed in table 6.6 – the following abbreviation applies to a configuration type.

Table 6.5. Notation used for the HCR configurations in the result tables.

Label	Join point	Description
—	–	No HCR branch
linear	sum at logit	Precomputed dependence score, all parent types; one learned scalar per endpoint, no encoder
nonlin. all 42D	sum at logit	42 evidence channels, all parent types; shared pair encoder
nonlin. bin. 10D	sum at logit	10 evidence channels, binary parents only; shared pair encoder
nonlin. bin. 42D	sum at logit	42 evidence channels, binary parents only; shared pair encoder
triple	sum at logit	13 channels of third-order moments over grandparent–parent–endpoint paths, replacing the pairwise evidence
typed+triple	concatenation	42 pairwise channels split across four non-shared encoders, one per parent type, each with a learned per-endpoint gate, plus a third-order block with its own per-endpoint gate column

Without HCR the model degrades sharply with depth and predicts a constant at $L = 6$. Every HCR variant recovers a substantial part of the lost performance, and the gain grows with depth: +1.8, +42.5 and +74.5 percentage points of the oracle AUC (eq. (12)) at $L = 1$, 3 and 6 respectively. The concatenated variant is the only one that remains flat across all six depths.

The reason for improved results with HCR is that HCR branch reads the dependence statistics of the direct parents and grandparents from a precomputed tensor and never passes through a graph convolution, so its contribution is identical at $L = 1$ and at $L = 6$.

Residual connections address the same failure from the opposite side: they preserve the signal inside the propagation channel instead of bypassing it. Based on results of experiments, we can observed that both (HCR and residual connections) are effective, but they are not complementary. Comparing models with enabled residual connections, no HCR variant improves on the model without HCR (table 6.7), and combining the two does not yield models alone. Once the propagation path with residual connections is repaired HCR does not provide any contribution.

Table 6.7. HCR variants on TransformerConv *with* residual connections.

		Validation				Test			
L	HCR	AUC	AP	Lift	% ceil.	AUC	AP	Lift	% ceil.
1	—	0.623	0.1255	2.31	81.30	0.619	0.1202	2.21	79.02
1	nonlin. bin.	0.622	0.1238	2.27	81.12	0.617	0.1176	2.16	77.82
1	nonlin. 42D	0.621	0.1250	2.30	80.56	0.616	0.1186	2.18	76.95
1	typed+triple	**0.631**	0.1335	2.45	**86.94**	**0.621**	0.1240	2.28	**80.30**
2	typed+triple	0.631	0.1322	2.43	86.77	0.619	0.1230	2.26	79.05
3	—	0.628	0.1346	2.47	85.23	0.620	0.1233	2.27	79.66
3	nonlin. bin.	0.627	0.1339	2.46	84.06	0.617	0.1242	2.28	77.48
3	nonlin. 42D	0.630	0.1343	2.47	85.95	0.619	**0.1247**	2.29	78.80
3	typed+triple	0.629	0.1350	2.48	85.64	0.618	0.1246	2.29	78.05
4	typed+triple	0.627	**0.1367**	**2.51**	84.10	0.621	**0.1256**	**2.31**	80.08
5	typed+triple	0.626	0.1361	2.50	83.88	**0.623**	**0.1256**	**2.31**	**81.54**
6	—	0.627	0.1337	2.46	84.19	0.620	0.1219	2.24	79.37
6	nonlin. bin.	0.614	0.1177	2.16	75.90	0.605	0.1082	1.99	69.84
6	nonlin. 42D	0.624	0.1301	2.39	82.21	0.618	0.1214	2.23	78.23
6	typed+triple	0.629	0.1347	2.47	85.62	0.620	0.1244	2.29	79.34

For understanding how HCR branch contributes to individual endpoints prediction, the learned gate weights are shown in fig. 6.1. Each cell gives the scalar by which the model scales the evidence block of one parent type before it enters the classification head; the gates were initialised at 1.0, so values below unity indicate reduced reliance on that channel.

The gate ordering aligns with the number of endpoints that have at least one direct parent of each type in the graph, suggesting that the model allocates more weight to parent types that are structurally more informative.

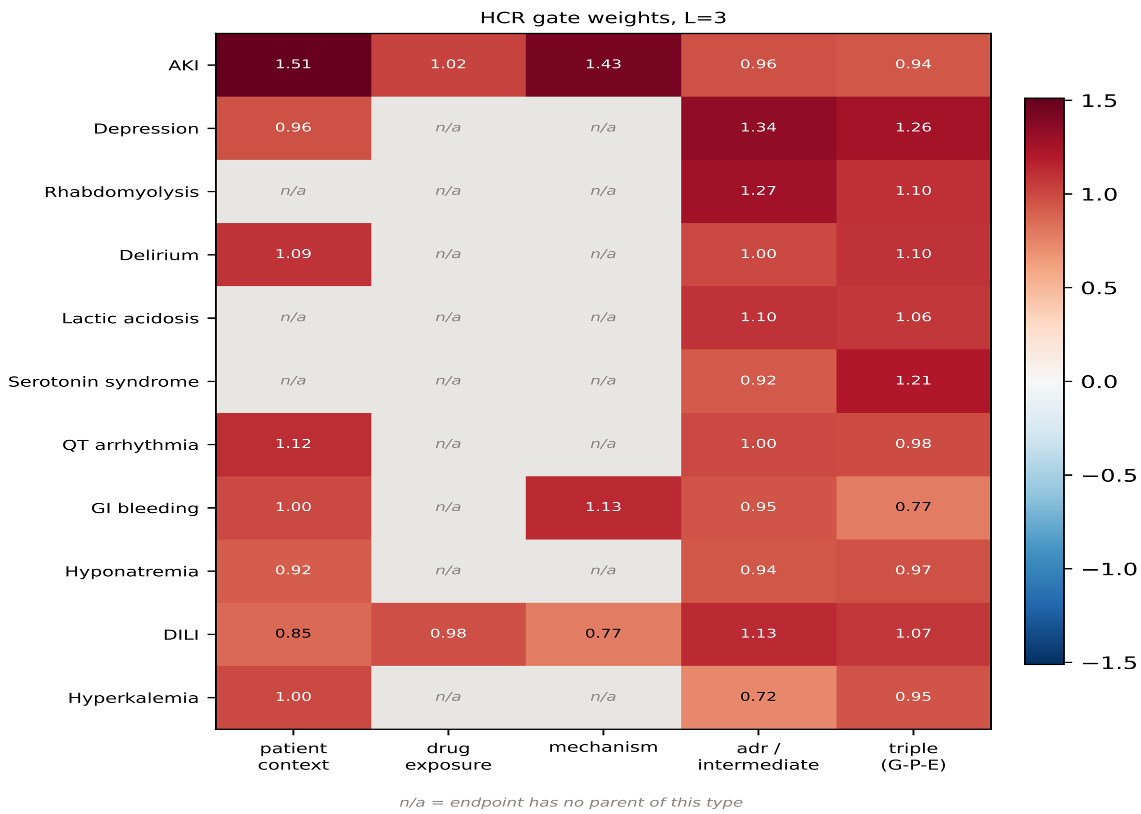

Figure 6.1. HCR gate matrix - influence of nodes type per endpoint

Taken together, the gate matrix indicates that the evidence branch is used unevenly across endpoints and parent types, in line with differences in parent availability and predictive utility on the synthetic task.

Taken together, it is worth to emphasize that all of models with HCR reached a satisfactory results comparable to models with residual connections applied. The gate matrix indicates that the evidence branch is used unevenly across endpoints and parent types, in line with differences in parent availability and predictive utility on the synthetic task. In general the value of the HCR branch therefore does not lie in raising the peak, but in bringing stability to learning acros all depths and additional information valid for the clinical event prediction. Across the six depths evaluated, *typed+triple* is the only variant that never collapses, retaining between 82% and 89% of the oracle AUC, while both the baseline and *nonlin. all 42D* fall below 80.00% at $L = 6$. The configuration *typed+triple* is mostly stable on depth and additionally bring the most informative value. The highest validation score in table 6.6 is obtained by *typed+triple* at $L = 1$ and $L = 4$ (89.05% and 86.65% of the oracle AUC). Bringing all together *typed+triple* HCR variant is chosen as the best.

6.2.3. Results per endpoint

Endpoints are ordered by prevalence. % ceil. is (12) computed against the per-endpoint generator-based oracle AUC; values above 100% should be read as saturation not Bayes-optimal limit.

7. Models benchmark

results for proxy dataset

7.1. Link prediction

7.2. Endpoint prediction

7.2.1. Data Representation

Task B is evaluated on a second benchmark dataset derived from Hetionet (section 4.2.2). The benchmark is a **structural proxy**—the pipeline, layer slots, and HCR branch are reused, but the graph is reconstructed to resemble the synthetic schema in topology.

Hetionet was selected because it is heterogeneous and supports a multi-label prediction setup analogous to the synthetic task. Adapting it nevertheless required a couple of reconstructions listed below.

(i) **Instance.** A compound takes the role of the patient: the topology is shared and a compound is described by the edges linking it to the remaining entities. A compound is therefore *not* a node of the graph, which means an entity can serve as a feature column only if it connects to compounds directly.

(ii) **Target.** The predicted endpoint is a disease node, labelled positive when a `treats` or `palliates` edge links the compound to that disease (the two relations were merged). Prevalences range from 1.29% to 4.51% over the twenty most frequent diseases.

(iii) **Layer mapping.** Node types were mapped onto the slots of the synthetic schema as presented in table 7.1.

(iv) **Direction.** Hetionet edges are undirected associations. For pipeline compatibility, every retained edge was oriented from the lower to the higher synthetic layer and intra-layer edges were discarded, guaranteeing acyclicity.

(v) **Path length.** Two changes lengthen the paths to a disease. First, a pathway and biological-process layer was inserted as the third layer, while the direct `DaG` edges were kept. Second, class-to-gene edges were added, because pharmacologic classes connect only to compounds in Hetionet and would otherwise have no edges at all. Such an edge is created when at least three training compounds of a class bind the same gene.

In fig. 7.1 there is presented the updated view of graph.

7.2.2. Results

The benchmark reproduces several trends observed on the synthetic dataset—depth sensitivity without stabilisers, partial recovery with HCR—under a different graph semantics and without oracle metrics. The gain from HCR is conditional on the state of the propagation path: at $L = 6$ without residual connections the baseline collapses to 0.585 AUC on test, while every HCR variant recovers a substantial part of it, up to 0.757 for *typed+triple*. At $L = 1$, where the receptive field of an endpoint coincides with the input of the HCR branch, no variant improves on the baseline, and the concatenated variants are slightly worse.

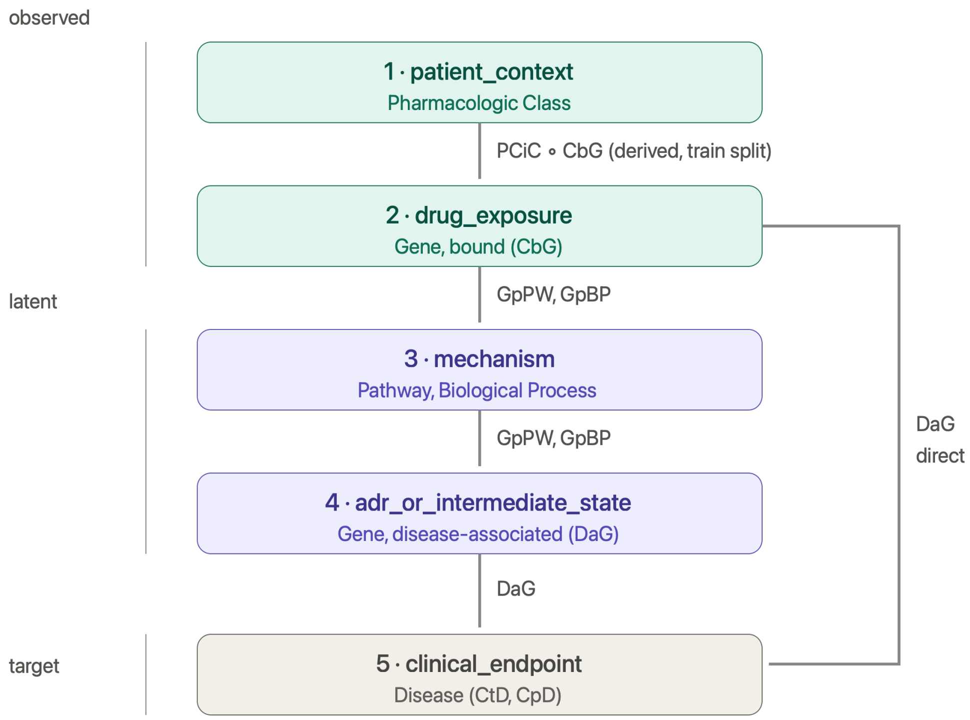

Figure 7.1. Hetionet layer mapping after reconstruction onto the synthetic schema.

In the benchmark dataset, residual connections and HCR combine additively at $L = 6$. The baseline reaches 0.585 without either mechanism, 0.680–0.757 with HCR alone, 0.728 with residual alone, and 0.760 with both. The combination is therefore better than either component in isolation.

The reason is the structure of the two graphs. In the synthetic graph the direct parents of an endpoint are observed, so the HCR branch and the one-hop receptive field read the same variables and the two mechanisms address the same bottleneck. In the Hetionet-derived graph depth is more crucial because some layers are latent. Information must therefore traverse several hops, and the residual path and the HCR bypass contribute at different points of that route.

Overall, the proxy experiment suggests that combining message passing with the statistical evidence branch remains useful with multi-hop reasoning. Whether an additional residual path helps or hinders depends on where the statistical evidence enters the model.

Table 7.2. Benchmark results for endpoint prediction. *nonlin. bin.* uses a 9-channel evidence vector restricted to binary parents; *nonlin. all* uses 42 channels with the full parent scope; *typed+triple* concatenates one encoder per parent type with a third-order block. Coverage of depths differs between variants.

			Validation			Test		
L	Res.	HCR	AUC	AP	Lift	AUC	AP	Lift
1	no	—	0.757	0.188	7.92	0.664	0.120	6.49
1	no	nonlin. bin.	0.763	0.207	8.75	0.657	0.120	6.47
1	no	nonlin. all	0.761	0.206	8.69	0.665	0.140	7.55
1	no	typed+triple	0.765	0.160	6.75	0.662	0.104	5.63
1	yes	—	0.756	0.196	8.26	0.669	0.106	5.72
1	yes	nonlin. bin.	0.756	0.201	8.49	0.670	0.135	7.31
1	yes	nonlin. all	0.758	0.206	8.69	0.664	0.135	7.32
1	yes	typed+triple	0.750	0.157	6.61	0.661	0.098	5.30
2	no	nonlin. all	0.763	0.263	11.10	0.694	0.173	9.34
2	no	typed+triple	0.778	0.263	11.10	0.677	0.175	9.46
2	yes	nonlin. all	0.760	0.224	9.44	0.684	0.154	8.31
2	yes	typed+triple	0.765	0.232	9.79	0.690	0.174	9.43
3	no	—	0.756	0.199	8.38	0.722	0.152	8.24
3	no	nonlin. bin.	0.805	0.275	11.62	0.686	0.169	9.14
3	no	nonlin. all	0.800	0.259	10.91	0.700	0.166	8.99
3	no	typed+triple	0.794	0.237	10.01	0.702	0.168	9.11
3	yes	—	0.799	0.225	9.48	0.685	0.166	8.97
3	yes	nonlin. bin.	0.814	0.235	9.90	0.714	0.183	9.92
3	yes	nonlin. all	0.807	0.259	10.91	0.677	0.174	9.39
3	yes	typed+triple	0.805	0.248	10.45	0.708	0.180	9.72
4	no	nonlin. all	0.757	0.230	9.70	0.678	0.151	8.14
4	no	typed+triple	0.807	0.262	11.06	0.747	0.181	9.79
4	yes	nonlin. all	0.814	0.238	10.05	0.715	0.180	9.76
4	yes	typed+triple	0.806	0.240	10.12	0.713	0.168	9.06
5	no	nonlin. all	0.783	0.239	10.08	0.680	0.158	8.52
5	no	typed+triple	0.807	0.246	10.37	0.746	0.163	8.84
5	yes	nonlin. all	0.815	0.291	12.29	0.727	0.179	9.71
5	yes	typed+triple	0.813	0.270	11.37	0.724	0.211	11.41
6	no	—	0.610	0.121	5.11	0.585	0.077	4.14
6	no	nonlin. bin.	0.782	0.210	8.86	0.680	0.174	9.42
6	no	nonlin. all	0.794	0.237	9.99	0.682	0.157	8.51
6	no	typed+triple	0.803	0.263	11.11	0.757	0.175	9.45
6	yes	—	0.817	0.255	10.77	0.728	0.173	9.36
6	yes	nonlin. bin.	0.832	0.310	13.06	0.760	0.179	9.70
6	yes	nonlin. all	0.812	0.235	9.92	0.734	0.166	8.96
6	yes	typed+triple	0.808	0.251	10.59	0.730	0.205	11.10

8. Conclusions

Podsumowanie i krytyczna analiza osiągniętych rezultatów. Ocena, czy osiągnięto założony cel pracy. Dyskusja co można było zrobić lepiej i propozycja dalszych prac w celu usprawieniania opracowanego rozwiązania.

Table 5.4. Fusion modes for the statistical evidence branch in Task B.

Mode	Evidence tensor
`linear`	Scalar HCR score $s_{p,v} = \sum_r a_{11,rv}\,\phi_1(R_p)$ per endpoint
`nonlinear`	Pairwise 10D or 42D vectors per (patient, parent, endpoint) slot
`typed_concat`	42D pairwise vectors, split into four type-specific tensors (`drug_exposure`, `mechanism`, `adr_or`
`triple`	13-channel vectors per (patient, triple, endpoint) slot along paths $G \to P \to Y$

Table 6.1. Hyperparameters shared by all architectures. Values were fixed across the experiments reported in this chapter

Group	Hyperparameter	Value
Encoder	Hidden units per convolutional layer	64
	Number of layers L	1, 2, 3, 4, 5, 6
	Aggregation across relations	sum
	Activation	ReLU
	Batch normalisation	yes
Regularisation	Dropout	0.2
	Weight decay	5×10^{-4}
Classification head	Number of layers	2
	Hidden units	64 (equal to encoder width)
	Output units	1 per endpoint
Optimisation	Optimiser	Adam
	Learning rate	1×10^{-3}
	Batch size	128
	Epochs (maximum)	200
	Loss	weighted BCE with logits
	Positive weight cap	30
	Gradient clipping	disabled
	Seed	42
Model selection	Selection metric	validation AP
	Early-stopping patience	20 epochs
	Minimum improvement	0.001

Table 6.6. HCR variants on TransformerConv *without* residual connections.

		Validation				Test			
L	HCR	AUC	AP	Lift	% ceil.	AUC	AP	Lift	% ceil.
1	—	0.622	0.1251	2.30	81.19	0.620	0.1197	2.20	79.61
1	linear 42D	0.622	0.1247	2.29	81.10	0.617	0.1191	2.19	77.83
1	nonlin. all 42D	0.622	0.1243	2.28	81.17	0.618	0.1192	2.19	78.57
1	typed+triple	0.634	**0.1348**	2.48	**89.05**	0.626	0.1241	2.28	**83.46**
2	nonlin. all 42D	0.625	0.1242	2.28	82.77	0.615	0.1180	2.17	76.15
2	typed+triple	0.628	0.1328	2.44	84.91	0.621	0.1235	2.27	80.16
3	—	0.550	0.0833	1.53	33.19	0.552	0.0734	1.35	34.49
3	linear 42D	0.604	0.1173	2.15	69.28	0.612	0.1031	1.90	74.51
3	triple	0.602	0.1131	2.08	67.92	0.590	0.1021	1.88	59.81
3	nonlin. all 42D	0.621	0.1310	2.41	80.13	0.629	0.1197	2.20	85.63
3	typed+triple	0.625	**0.1362**	2.50	**82.90**	0.620	**0.1252**	2.30	79.54
4	nonlin. all 42D	0.616	0.1214	2.23	76.90	0.609	0.1168	2.15	72.30
4	typed+triple	0.631	0.1350	2.48	86.65	0.614	0.1237	2.27	75.53
5	nonlin. all 42D	0.617	0.1183	2.17	77.67	0.611	0.1132	2.08	73.37
5	typed+triple	0.627	0.1320	2.42	84.31	0.617	0.1225	2.25	77.58
6	—	0.527	0.0653	1.20	17.96	0.500	0.0544	1.00	0.00
6	linear 42D	0.616	0.1206	2.21	76.91	0.603	0.0955	1.76	68.22
6	triple	0.607	0.1154	2.12	70.71	0.579	0.0967	1.78	52.56
6	nonlin. all 42D	0.569	0.0955	1.75	45.53	0.500	0.0544	1.00	0.00
6	typed+triple	0.630	0.1326	2.43	86.53	0.618	0.1218	2.24	78.35

Table 6.8. Per-endpoint results of the *typed+triple* configuration at $L = 4$ without residual connections.

		Oracle		Validation			Test		
Endpoint	Prev.	AUC	AP	AUC	AP	% ceil.	AUC	AP	% ceil.
AKI	0.131	0.811	0.454	0.794	0.406	94.3	0.785	0.421	91.6
Depression	0.085	0.652	0.166	0.655	0.165	101.6	0.639	0.170	91.0
Hyponatremia	0.079	0.638	0.168	0.613	0.132	81.5	0.607	0.142	77.5
Hyperkalemia	0.049	0.590	0.083	0.595	0.080	105.7	0.550	0.074	55.2
Delirium	0.037	0.668	0.100	0.649	0.093	89.0	0.687	0.098	93.0
GI bleeding	0.021	0.570	0.080	0.570	0.080	99.0	0.528	0.029	39.6
DILI	0.018	0.653	0.086	0.563	0.067	41.3	0.579	0.036	51.6
QT arrhythmia	0.016	0.622	0.075	0.591	0.050	74.1	0.536	0.019	29.4
Macro	0.055	0.651	0.151	0.629	0.134	85.4	0.614	0.124	75.5
Trained but excluded from the macro average									
Lactic acidosis	0.0057	0.523	0.011	0.560	0.009	–	0.549	0.008	–
Rhabdomyolysis	0.0037	0.530	0.011	0.536	0.005	–	0.724	0.033	–
Serotonin syndrome	0.0027	0.554	0.008	0.646	0.010	–	0.642	0.008	–

Table 7.1. Mapping of Hetionet entities onto the synthetic layer schema.

Layer	Slot	Hetionet entity	Observed
1	`patient_context`	Pharmacologic Class (filtered)	yes
2	`drug_exposure`	Gene, bound (`CbG`)	yes
3	`mechanism`	Pathway, Biological Process	no (latent)
4	`adr_or_intermediate_state`	Gene, associated (`DaG`)	no (latent)
5	`clinical_endpoint`	Disease	target

9. Appendix

Variable classes and generation mechanisms

Table 9.1. Variable classes in the primary synthetic patient-level dataset.

Variable class	Number	Value type	Generation mechanism and role
`patient_id`	1	Integer	Unique technical patient identifier; not a graph node and not used as a predictive feature.
`hospital_id`	1	Categorical integer	Technical identifier of a simulated hospital. The primary `clean` scenario contains one hospital; multiple hospitals are used only in the `multihospital` extension.
`patient_context`	24	Mostly binary; selected count or baseline-measurement variables	Patient demographics, comorbidities, baseline clinical characteristics, latent severity, and treatment propensity. Values are initialised before downstream graph variables.
`drug_exposure`	29	Binary	Stochastic indicators of exposure to a drug class. Their probabilities are generated by the additive logistic model conditional on their direct graph parents.
`mechanism`	49	Binary, count, and deterministic gate variables	Physiological mechanisms, interaction gates, drug-load variables, and other mediating concepts. Values are either sampled by the additive logistic model or calculated deterministically from their components.

Continued on next page

Variable class	Number	Value type	Generation mechanism and role
adr_or_intermediate_state	28	Mostly binary; one derived score and one threshold variable	Intermediate adverse states and physiological consequences. Most are generated stochastically; cumulative_adr_burden and severe_adr_burden are deterministic derived variables.
observation_or_selection	12	Binary	Variables representing testing, recording, reporting, or selection processes rather than the occurrence of a clinical state.
clinical_endpoint	13	Binary	Downstream clinical outcomes. Each is sampled from a Bernoulli distribution using a patient-specific probability conditional on its direct parents. These variables form the target labels for endpoint prediction.

Deterministic variables

Table 9.2. Deterministic variables and their generating rules.

Variable	Rule type	Generating rule
active_drug_count	Drug count	Sum of all 29 binary drug_exposure variables.
ddi_renal_double_hit	Conjunctive gate	nsaid $\wedge$ acei_arb.
ddi_renal_triple_whammy	Conjunctive gate	nsaid $\wedge$ acei_arb $\wedge$ diuretic.
ddi_cns_depression_synergy	Conjunctive gate	opioid $\wedge$ benzodiazepine.

Continued on next page

Variable	Rule type	Generating rule
ddi_bleeding_dual	Conjunctive gate	anticoagulant ∧ antiplatelet.
ddi_bleeding_triple	Conjunctive gate	nsaid ∧ anticoagulant ∧ antiplatelet.
ddi_serotonergic_synergy	Conjunctive gate	ssri ∧ snri.
ddi_hepatic_triple_hit	Conjunctive gate	hepatotoxic_drug_A ∧ antibiotic_hepatic_risk ∧ valproate_like_drug.
ddi_qt_multidrug_load	Drug count	qt_prolonging_drug + ssri + snri + macrolide + azole_antifungal.
nephrotoxin_load	Drug count	nsaid + aminoglycoside + contrast_agent + lithium + acei_arb.
hepatic_drug_load	Drug count	hepatotoxic_drug_A + antibiotic_hepatic_risk + valproate_like_drug + statin + macrolide + azole_antifungal.
qt_drug_load_v3	Drug count	qt_prolonging_drug + ssri + snri + macrolide + azole_antifungal + digoxin.
cns_depressant_load	Drug count	opioid + benzodiazepine + anticholinergic + valproate_like_drug.
bleeding_risk_load	Drug count	anticoagulant + antiplatelet + nsaid + ssri + corticosteroid.
serotonergic_load	Drug count	ssri + snri + opioid + triptan.
cyp_inhibitor_load	Drug count	macrolide + azole_antifungal.
ddi_statin_cyp_inhibitor	Conjunctive gate	statin ∧ $\mathbb{1}$(cyp_inhibitor_load > 0).
ddi_metformin_renal_risk	Conjunctive gate	metformin ∧ ckd.

Continued on next page

Variable	Rule type	Generating rule
`ddi_lithium_renal_risk`	Conjunctive gate	`lithium` $\wedge$ `ckd`.
`ddi_digoxin_electrolyte_risk`	Conjunctive gate	`digoxin` $\wedge$ `electrolyte_disturbance`.
`drug_disease_nsaid_ckd`	Conjunctive gate	`nsaid` $\wedge$ `ckd`.
`drug_disease_nsaid_heart_failure`	Conjunctive gate	`nsaid` $\wedge$ `heart_failure`.
`drug_disease_qt_baseline_risk`	Conjunctive gate	$\mathbb{1}$(`qt_drug_load_v3` $>$ 0) $\wedge$ `baseline_qt_risk`.
`drug_disease_hepatic_liver_disease`	Conjunctive gate	$\mathbb{1}$(`hepatic_drug_load` $>$ 0) $\wedge$ `liver_disease`.
`drug_disease_anticoagulant_frailty`	Conjunctive gate	`anticoagulant` $\wedge$ `frailty`.
`ddi_serotonergic_high_load`	Threshold	$\mathbb{1}$(`serotonergic_load` $\geq$ 2).
`cumulative_adr_burden`	Weighted sum	$\sum_k \omega_k \cdot \mathtt{ADR}_k + 0.08 \cdot$ `active_drug_count`, where ω_k is the fixed severity weight for adverse-state component k.
`severe_adr_burden`	Threshold	$\mathbb{1}$(`cumulative_adr_burden` $\geq$ 4.0).
`elevated_creatinine_recorded`	Recorded-state rule	`creatinine_rise` $\wedge$ `creatinine_tested`.
`elevated_alt_ast_recorded`	Recorded-state rule	`alt_ast_rise` $\wedge$ `lft_tested`.
`qt_recorded`	Recorded-state rule	`qt_prolongation_state` $\wedge$ `ecg_performed`.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. DOI: 10.1109/CVPR.2016.91

[2] A. Hogan et al., *Knowledge Graphs* (Synthesis Lectures on Data, Semantics, and Knowledge). Morgan & Claypool Publishers, 2021. DOI: 10.2200/S00367ED1V01Y202102WBE024

[3] J. Barrasa, M. Natarajan, and J. Webber, *Building Knowledge Graphs: A Practitioner's Guide*. O'Reilly Media, 2023.

[4] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer", in *Proceedings of The Web Conference (WWW)*, 2020, pp. 2704–2710. DOI: 10.1145/3366423.3380027

[5] F. MacLean, "Knowledge graphs and their applications in drug discovery", *Expert Opinion on Drug Discovery*, 2021. DOI: 10.1080/17460441.2021.1910673

[6] S. Toonsi, P. N. Schofield, and R. Hoehndorf, "Causal knowledge graph analysis identifies adverse drug effects", *arXiv preprint arXiv:2505.06949*, 2025.

[7] K. Lyu et al., "Causal knowledge graph construction and evaluation for clinical decision support of diabetic nephropathy", *Journal of Biomedical Informatics*, 2023. DOI: 10.1016/j.jbi.2023.104298

[8] M. Mesinovic, M. Buhlan, and T. Zhu, "Causal graph neural networks for healthcare", *Nature Biomedical Engineering*, vol. 10, pp. 1536–1556, 2026, Also available as arXiv:2511.02531. DOI: 10.1038/s41551-025-01531-2

[9] X. Wang, X. He, Y. Cao, M. Liu, and T.-S. Chua, "KGAT: Knowledge graph attention network for recommendation", in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 950–958. DOI: 10.1145/3292500.3330989

[10] F. Shehzad, M. Ferrari Dacrema, and D. Jannach, "A worrying reproducibility study of intent-aware recommendation models", in *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2025, pp. 3155–3164. DOI: 10.1145/3726302.3730220

[11] W. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs", in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 1024–1034.

[12] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks", in *The Semantic Web: 15th International Conference, ESWC 2018*, 2018, pp. 593–607. DOI: 10.1007/978-3-319-93417-4_38

[13] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks", in *International Conference on Learning Representations*, 2018.

[14] A. Vaswani et al., “Attention is all you need”, in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[15] F. Wójcik, *Grafowe sieci neuronowe. Teoria i praktyka*. Gliwice: Helion, 2026, 224 pp., ISBN: 978-83-289-3390-3.

List of Figures

List of Tables